

Module 1: Basics of Programming

Introduction to C++ Programming — A Beginner's Course

Module Overview

Students write their first real program today and, more importantly, learn to read what the compiler tells them when something goes wrong. The goal is not just "Hello, World!" — it's building the mental model of source code → compiler → executable that every later module leans on, and building comfort with compiler errors as normal, useful feedback rather than something to fear.

Suggested Duration: 60–75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain, in their own words, what a compiler does and why C++ needs one.
- Identify and explain every line of a minimal C++ program (`#include`, `main`, `cout`, semicolons, `return`).
- Compile and run a C++ program from the terminal without assistance.
- Read a compiler error message and locate the actual line responsible for it.
- Use single-line and multi-line comments appropriately.

Prerequisites

- Module 0 complete: working compiler, VS Code, and a pushed first commit.

Materials & Instructor Prep

- Projector with a large, high-contrast terminal font — students need to read error text from across the room.
- `01-basics-of-programming/examples/04_broken_missing_semicolon.cpp` preloaded, ready to compile live.

Key Vocabulary

Term	Definition
Machine code	The raw numeric instructions a CPU executes directly.
Compiled language	A language translated entirely to machine code before running (contrast: interpreted languages, run line-by-line).
Statement	One instruction in C++, always ending in a semicolon.
Entry point	The function where a program's execution begins — <code>main()</code> in every C++ program.
Namespace	A way of grouping names to avoid clashes; <code>std::</code> is the standard library's namespace.
Syntax error	A mistake that breaks C++'s grammar rules, caught by the compiler before the program ever runs.

Lesson Outline & Instructor Talking Points

1. The compilation pipeline, as a diagram (10 min)

- Draw source → compiler → executable → run on the board before showing any code. Ask: "Which of these three things is g++?"
- Contrast briefly with an interpreted language if students have any prior exposure (Python, Scratch) — "there, there's no separate build step; here, there always is."

Instructor note: Keep this abstract framing under 10 minutes — the concept lands better once they've seen it happen than while it's still just a diagram.

2. Anatomy of `hello.cpp`, line by line (20 min)

- Go line by line: `#include <iostream>` ("borrowing a toolbox"), `int main()` ("the door the program walks in through"), `std::cout << ... << std::endl;` (arrow metaphor: data flows into the stream, onto the screen), `return 0;` ("reporting back: I finished fine").
- Ask students to predict output before running any of the three examples in examples/ — prediction-before-execution is the single highest-leverage habit to build this early.
- Deliberately show what happens when `std::` is dropped without using namespace `std`; — connects back to the namespace vocabulary term.

Instructor note: Some students will ask why using namespace `std`; isn't used in this course's examples. Answer honestly: it's a real, common style choice — the course writes `std::` explicitly so it's always clear

where a name comes from, and both styles are worth recognizing.

3. Reading compiler errors as a skill, not a threat (20 min)

- Compile examples/04_broken_missing_semicolon.cpp live. Before scrolling to the error, ask the class to guess what's wrong just from having seen the file.
- Show the actual error message and highlight: the error often points to the line AFTER the real mistake — teach 'check the line above' as a standing habit.
- Fix it live, recompile, celebrate the clean build.

Instructor note: *This is the most important 20 minutes in the whole module for long-term student confidence. A student who leaves this class believing 'errors are normal and readable' will debug independently for the rest of the course. A student who leaves believing 'errors mean I'm bad at this' will not.*

4. Comments — what they're for and what they aren't (5–10 min)

- Show `//` and `/* */` styles. Make the point explicitly: comments explain WHY, not WHAT — a comment restating `cout << "Hi";` as `// print Hi` is not useful; a comment noting a non-obvious reason for a choice is.

Instructor note: *Keep this brief; it lands better as a running correction during code review of the exercise/project than as a standalone lecture.*

Common Student Misconceptions

Misconception	How to Address It
"A missing semicolon crashes the program."	Clarify it never compiles at all — this is a compile-time failure, not a runtime crash. Nothing 'ran' yet.
"The first error in a long list is the least important one."	Show, live, how fixing only the first error in a cascade often makes 4 other errors vanish on recompile.
"std::cout is a variable I create."	Clarify it's a pre-existing object from the iostream library representing the screen — nothing to declare, only to use.

Exercises & Module Project

Exercise: 01-basics-of-programming/exercises/exercise1.cpp — print name, hobby, and a chained-`<<` greeting. Project: an ASCII greeting card using only `std::cout` and comments (01-basics-of-programming/project/README.md).

Facilitation guidance: Pair students up for the exercise if time allows — reading someone else's chained << attempt is a fast way to internalize the syntax. Let the project be genuinely personal (their own name, their own message) — the payoff of Module 1 is emotional as much as technical: 'I built something that's mine.'

Discussion & Comprehension Check Questions

1. Why does C++ need a compiler when some other languages don't require one?

Guidance: Compiled languages translate everything ahead of time (often faster to run, extra build step); interpreted languages translate on the fly (simpler workflow, typically slower).

2. You see an error mentioning line 12, but line 12 looks correct. What's your first move?

Guidance: Check line 11 (or earlier) for a missing semicolon/brace — errors often surface one line late.

Assessment Ideas

- Quick quiz: given a 5-line broken program on a slide, have students identify the bug without a compiler, in writing.
- Ask students to explain `#include <iostream>` to a partner in one sentence, then swap and critique each other's explanation.

Answer Key Notes

See [01-basics-of-programming/exercises/solutions/exercise1_solution.cpp](#). Solutions are intentionally simple — the grading bar for this module is 'does it compile and produce personalized output,' not code elegance.

Wrap-Up & Bridge to Next Module

Bridge forward: "Every value we printed today was a literal, typed directly into the code. Next module, we give those values proper types and start storing them — that's the difference between a program that only prints fixed text and one that computes."

Instructor Tips

- Live-type everything in this module rather than pasting from slides — students need to see you make a typo and recover from it at least once.
- Keep terminal font size large (18pt+) when projecting; compiler error text is small and dense by default.